

# IN2033 IMPLEMENTATION REPORT

LEON SEFERI

## (EVERYTHING NEEDED FROM THE CODE AND TESTING FOR FINAL IMPLEMENTATION REPORT)

\*Section numbering is preliminary and subject to change according to the other content needed in Implementation report.

\*Will be compiled by another team member with other necessary content, forming our final report.

### SECTION 5.1 – SUBSYSTEM SET UP

IPOS-SA is implemented as a two-part web application, consisting of a REST API backend and single page application frontend. These are both designed to run locally on a single machine.

#### Backend – Python / FastAPI

The backend is written in Python 3.14+ using the FastAPI framework. Python is an interpreted language so requires no compilation step. Furthermore, the application is managed using uv, a Python package manager. This automatically resolves and installs all dependencies on first run.

The server can be started for the first time using the following:

**cd backend**

**pip install uv**

**uv sync**

**uv run python seed\_data\_demo.py # populate with demo data**

**uv run uvicorn main:app --reload**

The Uvicorn ASGI server (with FastAPI's standard install) handles all the HTTP requests. The database is SQLite in development – a single file (ipos\_sa.db) is created automatically in the backend directory on the first run.

PostgreSQL is supported for production through the DATABASE\_URL environment variable.

## Frontend – JavaScript/React/Vite

The frontend is written in JavaScript (ES modules) using the React 19 library. It is served using Vite 7 – a development server and build tool. Additionally, Node.js v20 or higher is needed.

The frontend can be started for the first time using:

**cd frontend**

**npm install**

**npm run dev**

\*Please note, to start application, open two terminals and start both backend and frontend.

The application runs on <http://localhost:5173>

For production deployment, npm run build bundles the frontend into static files in frontend/dist/.

### SECTION 5.1.1 – FILES CREATED

Python is an interpreted language so no compiled binary files (.exe, .jar, .class) are produced. The runtime consists of source files executed directly by the Python interpreter. The equivalent of a compiled executable is the combination of the source files and virtual environment installed by uv.

| File / Directory | Description                                                                                                                                                                                                                 | Location at Runtime |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|
| main.py          | Entry point for application. Initialises the FastAPI app and registers all routers. It also configures CORS and creates all of the database tables upon startup.                                                            | backend/            |
| pyproject.toml   | Python configuration file. Declares all backend dependencies with version constraints (FastAPI, SQLAlchemy, PyJWT etc) and the required Python version. Read by uv sync on first setup to install all packages into .venv/. | backend/            |

|                                                                                           |                                                                                                                                                                                                                                                              |                   |
|-------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|
| ipos_sa.db                                                                                | SQLite database file that is created automatically on first run, when running seed data script. Contains the users, orders, products and invoices.                                                                                                           | backend/          |
| .venv/                                                                                    | Python virtual environment containing all installed packages (FastAPI, SQLAlchemy, PyJWT, bcrypt). Created and managed by uv.                                                                                                                                | backend/.venv/    |
| auth/, catalogue/, merchants/, orders/, reports/, commercial_applications/, audit/, core/ | Feature modules where each contains models.py, router.py and service.py. These are executed by the Python Interpreter at runtime.                                                                                                                            | backend/<module>/ |
| seed_data_demo.py                                                                         | Script for populating the database with the official IPOS sample dataset prior to the demo. Run before starting backend.                                                                                                                                     | backend/          |
| package.json                                                                              | Node.js project configuration file. Declares all frontend dependencies with version pins and defines the npm scripts (npm install, npm run dev, npm run build). Npm install reads this file and generates package-lock.json to lock exact resolved versions. | frontend/         |
| frontend/dist/ (production only)                                                          | Static frontend files produced by npm run build. Contains index.html and hashed JS/CSS assets. Only needed if the Vite dev server is not used.                                                                                                               | frontend/dist/    |

In development and demo mode, the Vite dev server serves the frontend directly from source so no build step is required.

## SECTION 6.1 – COMMERCIAL OFF-THE-SHELF SOFTWARE COMPONENTS

Backend runtime dependencies that are installed automatically by uv.

| Component         | Version                         | Purpose                                                            |
|-------------------|---------------------------------|--------------------------------------------------------------------|
| Python            | 3.14+                           | Interpreter / runtime environment.                                 |
| FastAPI           | >= 0.128.5                      | Rest API web framework.                                            |
| Uvicorn           | Standard – Bundled with FastAPI | HTTP Server.                                                       |
| SQLModel          | >= 0.0.32                       | ORM – database models and queries. Built on SQLAlchemy + Pydantic. |
| Pydantic          | >= 2.12.5                       | Request/response data validation and serialisation.                |
| Pydantic-settings | >= 2.12.0                       | Configuration management via environment variables.                |
| PyJWT             | >= 2.11.0                       | JWT token generation and validation for authentication.            |
| bcrypt            | >= 5.0.0                        | Password hashing.                                                  |
| requests          | >= 2.32.5                       | HTTP client for outbound calls to IPOS-PU.                         |

Frontend runtime dependencies installed by npm install.

| Component        | Version | Purpose                                       |
|------------------|---------|-----------------------------------------------|
| Node.js          | V20+    | JavaScript runtime / package manager host.    |
| React            | 19.2.0  | UI component library.                         |
| React DOM        | 19.2.0  | DOM rendering for React.                      |
| React Router DOM | 7.13.0  | Client side routing between pages.            |
| React Icons      | 5.5.0   | Icon library (Feather icons used throughout). |
| Vite             | 7.3.1   | Development server and production build tool. |

## **SECTION 6.2 – DEPLOYMENT OF RUNTIME FILES**

All runtime components of IPOS-SA are deployed on the same computer. No remote database server or remote application server is used. The SQLite database is a local file within the project directory. Both the backend API (localhost:8000) and the frontend (localhost:5173) run as local processes on the same machine.

The other IPOS subsystems (IPOS-CA and IPOS-PU) are also deployed on the same machine for integration testing purposes.

IPOS-SA communicates with IPOS-PU via a REST endpoint for membership application notifications.

IPOS-CA communicates with IPOS-SA via the IPOS-SA REST API.

All inter subsystem communication occurs over localhost.

In conclusion, all runtime components of IPOS-SA (dependencies, database and other IPOS subsystems) are deployed on the same computer.

## SECTION 7.1 – UNIT TESTING OF IPOS-SA

Unit tests are written using Vitest (the JavaScript equivalent of JUnit). Tests are located in tests/services/ and run with:

**cd frontend**

**npm test -- --run**

The apiClient is mocked in all unit tests so that no running backend is required.

Two service modules (the JavaScript equivalent of classes) are covered. Each module covers more than 5 methods. orderService.js implements OrderAPIImpl while authService.js implements LoginAPIImpl and staff account management.

### **orderService.test.js (OrderAPIImpl – Test Cases 1-27)**

This wraps all order and catalogue API calls and exposes 6 functions (**placeOrder**, **trackOrderProgress**, **getOrderDetails**, **queryBalance**, **viewPreviousOrders**, **getCatalogue**). These are consistent with the class diagram generated in the first deliverable.

\*Test Case IDs are references to the created test cases from the first deliverable. Where N/A is written, this means it is additional coverage not specified in the first deliverable.

\*Suite number references are a count of the total number of tests created. They can also be used as references to find the specific test in our source code under tests/services/.

#### **placeOrder:**

| Suite: | Test Case ID | Parameter Values                | Description                          | Expected Outcome                             | Actual Outcome |
|--------|--------------|---------------------------------|--------------------------------------|----------------------------------------------|----------------|
| 1      | SA-ORD-01    | productId=101<br>quantity=10    | Valid product, valid quantity.       | Success=true, orderId and amountDue returned | Pass           |
| 2      | SA-ORD-02    | productId=200,<br>quantity=1    | Product does not exist in inventory. | Success=false, error returned                | Pass           |
| 3      | SA-ORD-05    | productId=100,<br>quantity = 0  | Boundary value – quantity of zero.   | success=false                                | Pass           |
| 4      | SA-ORD-06    | productId=100,<br>quantity = -1 | Negative quantity.                   | success=false                                | Pass           |

|   |           |                                           |                                                                |                                                         |      |
|---|-----------|-------------------------------------------|----------------------------------------------------------------|---------------------------------------------------------|------|
| 5 | SA-ORD-03 | productID = -1,<br>quantity = 1           | Invalid<br>product ID.                                         | success=false                                           | Pass |
| 6 | SA-ORD-04 | productID=100,<br>quantity =<br>1,000,000 | Extremely<br>large<br>quantity,<br>depends on<br>stock.        | Outcome is<br>Boolean and<br>both paths are<br>handled. | Pass |
| 7 | N/A       | productID =<br>101,<br>quantity = 2       | Checks if<br>correct<br>payload<br>shape is sent<br>to /orders | apiClient.post<br>called with {<br>items } array.       | Pass |

### trackOrderProgress:

| Suite | Test Case ID | Parameter Values                | Description                    | Expected Outcome                      | Actual Outcome |
|-------|--------------|---------------------------------|--------------------------------|---------------------------------------|----------------|
| 8     | SA-ORD-07    | orderId=order-1 (valid, exists) | Is it a valid order in system? | Returns status string e.g. 'accepted' | Pass           |
| 9     | SA-ORD-08    | orderId=99999 (does not exist)  | Order not found.               | Returns 'unknown'                     | Pass           |
| 10    | SA-ORD-09    | orderId=-1 (invalid)            | Invalid order ID.              | Returns 'unknown'                     | Pass           |
| 11    | SA-ORD-10    | orderId=0 (boundary)            | Boundary value.                | Returns 'unknown'                     | Pass           |

### GetOrderDetails:

| Suite | Test Case ID | Parameter Values              | Description                                                                      | Expected Outcome                                                                                                         | Actual Outcome |
|-------|--------------|-------------------------------|----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|----------------|
| 12    | N/A          | OrderID = 'order-abc' (valid) | Full order object returned. Verifies all snake_case backend fields are converted | merchantId = 'merch-1',<br>orderDate = '2026-01-15', discountAmount = 10.00, amountDue = 190.00, courierRef = 'DHL12345' | Pass           |

|    |     |                                       |                                                                      |                                                             |      |
|----|-----|---------------------------------------|----------------------------------------------------------------------|-------------------------------------------------------------|------|
|    |     |                                       | to camelCase                                                         |                                                             |      |
| 13 | N/A | OrderID = 'order-1' with items array. | Order with line items returned. Verifies items are correctly mapped. | Items[0].productName = 'Aspirin', items[0].unitPrice = 8.00 | Pass |
| 14 | N/A | OrderID = 'bad-id' (non existent)     | Order not found.                                                     | Throws 'Order not found' error.                             | Pass |

### queryBalance:

| Suite | Test Case ID | Parameter Values                                          | Description                                | Expected Outcome                                                                | Actual Outcome |
|-------|--------------|-----------------------------------------------------------|--------------------------------------------|---------------------------------------------------------------------------------|----------------|
| 15    | SA-ORD-11    | merchantID = 'merch-202' (valid, has outstanding balance) | Valid merchant with existing orders.       | Returns creditLimit = 5000.00, currentDebt = 1200.00, availableCredit = 3800.00 | Pass           |
| 16    | SA-ORD-12    | merchantID='merch-202' (valid, zero balance)              | Valid merchant with no outstanding orders. | Returns currentDebt = 0.00, availableCredit = 5000.00                           | Pass           |
| 17    | SA-ORD-13    | merchantID = '9999' (does not exist)                      | Merchant not found.                        | Throws error.                                                                   | Pass.          |
| 18    | SA-ORD-14    | merchantID = -1                                           | Invalid merchant ID.                       | Throws error.                                                                   | Pass.          |
| 19    | N/A          | merchantID = '0' (boundary value)                         | Boundary case – zero ID.                   | Throws error.                                                                   | Pass.          |

### viewPreviousOrders:

| Suite | Test Case ID | Parameter Values | Description | Expected Outcome | Actual Outcome |
|-------|--------------|------------------|-------------|------------------|----------------|
|-------|--------------|------------------|-------------|------------------|----------------|

|    |           |                                                  |                                                     |                                                                 |       |
|----|-----------|--------------------------------------------------|-----------------------------------------------------|-----------------------------------------------------------------|-------|
| 20 | SA-ORD-15 | merchantID = 'm1', has 2 existing orders.        | Merchant exists and has placed orders.              | Returns list of 2 order summaries.                              | Pass  |
| 21 | SA-ORD-16 | merchantID='m1' but has no orders.               | Merchant exists but no order history.               | Returns empty list.                                             | Pass  |
| 22 | SA-ORD-17 | merchantID = 2000 (does not exist)               | Merchant not found.                                 | Throws error.                                                   | Pass. |
| 23 | SA-ORD-18 | merchantID = -1 (invalid)                        | Invalid merchant ID                                 | Throws error.                                                   | Pass. |
| 24 | N/A       | merchantID = 'm1', single order with all fields. | Verifies camelCase field mapping on list responses. | orderDate = '2026-02-01', discountAmount = 15, amountDue = 285. | Pass  |

### getCatalogue:

| Suite | Test Case ID | Parameter Values                                           | Description                           | Expected Outcome                                                                            | Actual Outcome |
|-------|--------------|------------------------------------------------------------|---------------------------------------|---------------------------------------------------------------------------------------------|----------------|
| 25    | SA-ORD-19    | No parameters. Catalogue is pre-populated with 2 products. | Product exists in catalogue.          | Returns list of 2 products. productCode = 'ASP001', packageCost = 8.50, minStockLevel = 50. | Pass           |
| 26    | SA-ORD-20    | No parameters. Empty catalogue mock.                       | No products in catalogue.             | Returns empty array[]. Array.isArray result is true.                                        | Pass           |
| 27    | N/A          | No parameters. Backend throws Error ('Server error').      | Network or server error during fetch. | Returns [] instead of throwing. Page still renders without crashing.                        | Pass.          |

orderService unit test result: 27/27 passed.

```
✓ tests/services/orderService.test.js (27 tests) 11ms
```

**authService.test.js (LoginAPIImpl – Test Cases 28-56):**

**merchantLogin:**

| Suite | Test Case ID | Parameter Values                                                                                                                           | Description                                                                   | Expected Outcome                                                      | Actual Outcome |
|-------|--------------|--------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|-----------------------------------------------------------------------|----------------|
| 28    | SA-LOG-01    | username =<br>"merchant1",<br>password =<br>"correctPassword".                                                                             | Valid credentials.                                                            | success = true,<br>user returned,<br>JWT stored via<br>a setAuthToken | Pass           |
| 29    | SA-LOG-02    | username =<br>"merchant1",<br>password =<br>"wrongPassword".                                                                               | Wrong password.                                                               | success =<br>false, error<br>returned.                                | Pass           |
| 30    | SA-LOG-03    | username =<br>"unknown",<br>password =<br>"pass".                                                                                          | Merchant<br>does not<br>exist.                                                | success =<br>false, error<br>returned.                                | Pass           |
| 31    | SA-LOG-04    | username = "",<br>password =<br>"pass"                                                                                                     | Empty<br>username.                                                            | success =<br>false.                                                   | Pass.          |
| 32    | SA-LOG-05    | username =<br>"merchant1",<br>Password = "".                                                                                               | Empty<br>password.                                                            | success =<br>false.                                                   | Pass.          |
| 33    | SA-LOG-06    | username = "",<br>password = "".                                                                                                           | Both fields<br>empty.                                                         | success =<br>false.                                                   | Pass           |
| 34    | N/A          | username =<br>"merchant1",<br>password =<br>"correctPassword".<br>Backend returns {<br>message 'ok' }<br>with no<br>access_token<br>field. | Verifies a<br>response<br>missing the<br>token is<br>treated as a<br>failure. | Returns<br>success =<br>false.                                        | Pass           |

|    |     |                                                                                |                                                                                               |                                                                                                 |      |
|----|-----|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|------|
| 35 | N/A | username =<br>“merchant1”,<br>password =<br>“pass” and the<br>backend rejects. | Verifies any<br>existing<br>token is<br>cleared<br>before each<br>login, even<br>on failure.  | clearAuthToken<br>called.                                                                       | Pass |
| 36 | N/A | username =<br>“merchant1”,<br>password =<br>“pass123”.                         | Verifies the<br>correct<br>field names<br>and values<br>are sent to<br>the login<br>endpoint. | POST to<br>/auth/login<br>with body {<br>username:<br>“merchant1”,<br>password:<br>“pass123” }. | Pass |

### merchantDisconnect:

| Suite | Test Case ID | Parameter Values | Description                                               | Expected Outcome                                           | Actual Outcome |
|-------|--------------|------------------|-----------------------------------------------------------|------------------------------------------------------------|----------------|
| 37    | SA-LOG-07    | (valid session)  | Merchant<br>logged in,<br>logout<br>requested.            | Returns<br>true and<br>token<br>cleared.                   | Pass           |
| 38    | SA-LOG-08/09 | (backend errors) | Session not<br>found or<br>merchant<br>does not<br>exist. | Returns<br>false and<br>token still<br>cleared<br>locally. | Pass           |
| 39    | SA-LOG-10    | -                | Correct<br>logout<br>endpoint<br>hit.                     | POST<br>/auth/logout<br>called.                            | Pass.          |

### getCurrentUser:

| Suite | Test Case ID | Parameter Values                                                                                       | Description                                                                    | Expected Outcome                                                      | Actual Outcome |
|-------|--------------|--------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|-----------------------------------------------------------------------|----------------|
| 40    | N/A          | No input<br>parameters.<br>Backend returns<br>user object { id:<br>‘abc’, username:<br>‘admin1’, role: | Valid<br>authenticated<br>session.<br>Verifies pass<br>through to<br>/auth/me. | Returns user<br>object with all<br>fields. GET<br>/auth/me<br>called. | Pass           |

|    |     |                                                                |                                      |                              |      |
|----|-----|----------------------------------------------------------------|--------------------------------------|------------------------------|------|
|    |     | 'admin', email:<br>'admin@test.com'.                           |                                      |                              |      |
| 41 | N/A | No input parameters.<br>Backend throws Error ('Unauthorized'). | Session expired or no token present. | Throws 'Unauthorized error'. | Pass |

### changePassword:

| Suite | Test Case ID | Parameter Values                                                            | Description                                              | Expected Outcome                                                                            | Actual Outcome |
|-------|--------------|-----------------------------------------------------------------------------|----------------------------------------------------------|---------------------------------------------------------------------------------------------|----------------|
| 42    | N/A          | currentPassword = 'oldPass123',<br>newPassword = 'newPass456'               | Valid passwords, correct current password supplied.      | Returns success = true.                                                                     | Pass           |
| 43    | N/A          | currentPassword = 'wrongOldPass',<br>newPassword = 'newPass456'.            | Wrong current password supplied.                         | Returns success = false, error message contains 'incorrect'.                                | Pass           |
| 44    | N/A          | currentPassword = 'oldPass123',<br>newPassword = '123' (only 3 characters). | New password too short.                                  | Returns success = false.                                                                    | Pass.          |
| 45    | N/A          | currentPassword = 'oldPass',<br>newPassword = 'newPass'.                    | Verifies correct snake_case field names sent to backend. | POST to /auth/change-password with body {current_password, new_password}.<br>Not camelCase. | Pass           |

### getStaffUsers:

| Suite | Test Case ID | Parameter Values     | Description           | Expected Outcome         | Actual Outcome |
|-------|--------------|----------------------|-----------------------|--------------------------|----------------|
| 46    | N/A          | No input parameters. | Staff list retrieved. | Returns list of 2 users. | Pass           |

|    |     |                                                          |                                                             |                                                       |      |
|----|-----|----------------------------------------------------------|-------------------------------------------------------------|-------------------------------------------------------|------|
|    |     | Backend returns list of 2 users with is_active field.    | Verifies snake_case to camelCase mapping on list responses. | users[0].isActive = true, users[0].username='admin1'. |      |
| 47 | N/A | No input parameters. Backend throws Error ('Forbidden'). | Unauthorized role attempts to access staff list.            | Throws 'Forbidden' error.                             | Pass |

### createStaffUser:

| Suite | Test Case ID | Parameter Values                                                                                         | Description                   | Expected Outcome                                              | Actual Outcome |
|-------|--------------|----------------------------------------------------------------------------------------------------------|-------------------------------|---------------------------------------------------------------|----------------|
| 48    | N/A          | username = 'manager2',<br>email = 'mgr@test.com',<br>password = 'pass123',<br>role = 'manager'.          | Valid new staff user details. | Returns success = true,<br>Result.user.username = 'manager2'. | Pass           |
| 49    | N/A          | username = 'admin1' (already exists), email = 'taken@test.com',<br>password = 'pass',<br>role = 'admin'. | Username already in use.      | Returns success = false, error returned.                      | Pass           |

### changeUserRole:

| Suite | Test Case ID | Parameter Values                         | Description                               | Expected Outcome                                          | Actual Outcome |
|-------|--------------|------------------------------------------|-------------------------------------------|-----------------------------------------------------------|----------------|
| 50    | N/A          | userId = '2',<br>role = 'director'.      | Valid role change for an existing user.   | Returns success = true,<br>result.user.role = 'director'. | Pass           |
| 51    | N/A          | userId = 'self-id',<br>role = 'manager'. | Attempting to change own role.<br>Backend | Returns success = false.                                  | Pass           |

|    |     |                                  |                                             |                                                              |      |
|----|-----|----------------------------------|---------------------------------------------|--------------------------------------------------------------|------|
|    |     |                                  | prevents self-demotion.                     |                                                              |      |
| 52 | N/A | userId = '5',<br>role = 'admin'. | Verifies correct endpoint and request body. | PATCH /auth/users/5/role called with body { role: 'admin' }. | Pass |

### deleteStaffUser:

| Suite | Test Case ID | Parameter Values             | Description                                                       | Expected Outcome                                                            | Actual Outcome |
|-------|--------------|------------------------------|-------------------------------------------------------------------|-----------------------------------------------------------------------------|----------------|
| 53    | N/A          | userId = 'user-id-123'       | Valid staff user deletion. Backend returns 204 (null body).       | Returns success = true.                                                     | Pass           |
| 54    | N/A          | userId = 'nonexistent-id'    | User does not exist in system.                                    | Returns success = false, error returned.                                    | Pass           |
| 55    | N/A          | userId = 'merchant-user-id'. | Merchant account targeted via staff endpoint. Backend rejects it. | Returns success = false. Merchant accounts must use the merchants endpoint. | Pass           |
| 56    | N/A          | userId = 'abc-123'.          | Verifies correct endpoint called.                                 | DELETE /auth/users/abc-123 called.                                          | Pass           |

authService unit test result: 29/29 passed.

```
✓ tests/services/authService.test.js (29 tests) 9ms
```

Overall unit test results: 56/56 passed.

## SECTION 7.2 SUBSYSTEM AND SYSTEM TESTING

### 7.2.1 Subsystem Testing – Provided Interface Tests

IPOS-SA provides two interfaces used by external subsystems:

ISALoginAPI is used by IPOS-CA to authenticate merchants.

ISAOOrderAPI is used by IPOS-CA to place and track orders. One method from each provided interface is tested.

#### ISALoginAPI.merchantLogin in merchantLogin.test.js (Suites 57-68):

| Suite | Test Case ID | Parameter Values                                     | Description                                                  | Expected Outcome                               | Actual Outcome |
|-------|--------------|------------------------------------------------------|--------------------------------------------------------------|------------------------------------------------|----------------|
| 57    | SA-LOG-01    | username = "merchant1",<br>password = "correctPass". | Valid credentials.                                           | success = true, user object returned.          | Pass           |
| 58    | SA-LOG-02    | username = "merchant1",<br>password = "wrongPass".   | Wrong password.                                              | success = false, error returned.               | Pass           |
| 59    | SA-LOG-03    | username = "unknown",<br>password = "pass".          | Merchant does not exist.                                     | success = false, error returned.               | Pass           |
| 60    | SA-LOG-04    | username = "",<br>password = "pass"                  | Empty username.                                              | success = false, error returned.               | Pass.          |
| 61    | SA-LOG-05    | username = "merchant1",<br>Password = "".            | Empty password.                                              | success = false, error returned.               | Pass.          |
| 62    | SA-LOG-06    | username = "",<br>password = "".                     | Both fields empty.                                           | success = false, error returned.               | Pass           |
| 63    | N/A          | username = "merchant1",<br>password = "correctPass". | Verifies that the correct HTTP method and endpoint are used. | POST request sent to /auth/login exactly once. | Pass           |
| 64    | N/A          | username = "merchant1",<br>password = "correctPass"  | Verifies the request body uses the correct                   | POST body contains { username: "merchant1",    | Pass           |

|    |     |                                                                                                       |                                                                      |                                                         |      |
|----|-----|-------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|---------------------------------------------------------|------|
|    |     |                                                                                                       | field names.                                                         | password: "correctPass" }                               |      |
| 65 | N/A | username = "merchant1",<br>password = "correctPass".                                                  | Verifies the JWT token returned in the response is stored.           | setAuthToken called with 'valid-jwt-token'.             | Pass |
| 66 | N/A | username = "merchant1",<br>password = "correctPass"<br>(backend rejects).                             | Verifies any existing token is cleared before each login attempt.    | clearAuthToken called before the request.               | Pass |
| 67 | N/A | username = "merchant1",<br>password = "wrongPass".                                                    | Verifies no token is stored when login fails.                        | setAuthToken not called.                                | Pass |
| 68 | N/A | username = "merchant1",<br>password = "correctPass",<br>Response body contains no access_token field. | Verifies a response missing the token field is treated as a failure. | Returns success = false and setAuthToken is not called. | Pass |

**Result: 12/12 passed.**

```
✓ tests/services/merchantLogin.test.js (12 tests) 3ms
```

**ISAOrderAPI.placeOrder in placeOrder.test.js (Suites 69-78):**

| Suite | Test Case ID | Parameter Values                  | Description                    | Expected Outcome | Actual Outcome |
|-------|--------------|-----------------------------------|--------------------------------|------------------|----------------|
| 69    | SA-ORD-01    | productID = 1001,<br>quantity = 5 | Valid product, valid quantity. | success = true.  | Pass           |
| 70    | SA-ORD-02    | productID = 9999,<br>quantity = 5 | Product does not exist.        | success = false. | Pass           |

|    |           |                                                                                                                   |                                                                        |                                                                  |       |
|----|-----------|-------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|------------------------------------------------------------------|-------|
| 71 | SA-ORD-05 | productID = 1001,<br>quantity = 0                                                                                 | Boundary value – zero quantity.                                        | success = false.                                                 | Pass  |
| 72 | SA-ORD-06 | productID = 1001,<br>quantity = -1                                                                                | Negative quantity.                                                     | success = false.                                                 | Pass. |
| 73 | SA-ORD-03 | productID = -1,<br>quantity = 5                                                                                   | Invalid product ID.                                                    | success = false.                                                 | Pass. |
| 74 | SA-ORD-04 | productID = 1001,<br>quantity = 1000000                                                                           | Extremely large quantity.                                              | Outcome is boolean regardless of stock level.                    | Pass  |
| 75 | N/A       | productID = 1001,<br>quantity = 5                                                                                 | Verifies the correct HTTP method and endpoint are used.                | POST request sent to /orders exactly once.                       | Pass  |
| 76 | N/A       | Items = [{ productid:1001,<br>quantity:5}]                                                                        | Verifies the items array is sent in the request body.                  | POST body contains { items: [{ productid: 1001, quantity: 5 }] } | Pass  |
| 77 | N/A       | productID = 1001,<br>quantity = 5.<br>Backend returns order_id = 'ORD-001',<br>amount_due = 90,<br>discount = 10. | Verifies snake_case response fields are correctly mapped to camelCase. | result.orderId = 'ORD-001',<br>result.amountDue = 90             | Pass  |
| 78 | N/A       | productID = 1001,<br>quantity = 1.<br>Backend throws Error ('Network error').                                     | Verifies a network error returns a failure object and does not throw.  | Returns success = false. Does not throw.                         | Pass  |

**Result: 10/10 passed.**

```
✓ tests/services/placeOrder.test.js (10 tests) 7ms
```

**Overall subsystem test result: 22/22 passed.**

### 7.2.2 System Testing – Required Interface

IPOS-SA uses IPUCommsAPI provided by IPOS-PU to send email notifications to applicants following a commercial membership decision. The method under test is `sendEmail(email, body, subject)`. This is accessed via the `notifyApplication` wrapper in `notificationService.js` in `frontend/services/`.

The brief requires tests for 2 methods of the required interface. IPUCommsAPI exposes only a single method: `sendEmail(email, body, subject)`. As this is the only method available on the required interface, all system tests cover this method.

Results are not required and can only be obtained during integration testing with the IPOS-PU team.

#### **notifyApplication.test.js:**

| Suite | Test Case ID | Parameter Values                                                                                                           | Description                        | Expected Outcome                                                                                                                               |
|-------|--------------|----------------------------------------------------------------------------------------------------------------------------|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| 79    | SA-IPU-01    | Email = <a href="mailto:application@test.com">application@test.com</a> ,<br>valid outcome = 'approved', valid company ref. | All valid parameters.              | Returns true. POST made to IPOS-PU with correct email, correct subject containing outcome, as well as a body containing the company reference. |
| 80    | SA-IPU-03    | Outcome = 'pending' (non-terminal state).                                                                                  | Pending – no email should be sent. | Returns false without calling <code>sendEmail</code> .                                                                                         |

|    |              |                                             |                                       |                                                                |
|----|--------------|---------------------------------------------|---------------------------------------|----------------------------------------------------------------|
| 81 | SA-IPU-04/05 | IPOS-PU unavailable<br>(connection refused) | Required<br>interface<br>unavailable. | Returns false.<br>An attempt<br>was made but<br>did not crash. |
| 82 | SA-IPU-06    | email = "", outcome =<br>'pending'.         | Completely<br>invalid input           | Returns false<br>without<br>calling<br>sendEmail.              |

### Overall test suite result:

**82/82 passed.**

```

Test Files 5 passed (5)
  Tests 82 passed (82)
Start at 09:40:16
Duration 3.08s (transform 161ms, setup 116ms, collect 269ms, tests 35ms, environment 12.45s, prepare 1.03s)

```

\*Please note, upon running the test suite, stderr lines are visible in the output. This is expected and they are internal error messages logged by the service functions when error handling paths are used by the mocked responses. They do not indicate test failures. All 82 tests pass.

## **SECTION 7.3 – NON-FUNCTIONAL TESTING CONSIDERATIONS**

The following describes how nonfunctional attributes of a production deployment of IPOS-SA could be tested. Actual results from these tests are not provided as they are not required at this stage.

### **Security:**

IPOS-SA exposes a login endpoint at POST /auth/login that accepts username and password credentials. A brute force test should be conducted to verify that the backend correctly rate limits or rejects repeated failed attempts from the same origin (within a short time window).

Additionally, all API endpoints that accept user supplied string input (such as order placement and merchant queries) should be tested for injection resistance. This risk is already reduced by SQLModel's use of parameterised queries internally, but the endpoints should be verified under automated security scanning.

JWT token expiry should be confirmed by attempting to use a token beyond its configured lifetime and verifying a 401 response is returned.

### **Scalability:**

Based on the staff structure and the number of registered merchants, realistic peak concurrent usage is estimated at 10-20 simultaneous users. A load testing tool like Locust or k6 could simulate 10 times this load to verify the system remains responsive. Acceptable thresholds would be under 500ms for write operations and under 200ms for read operations at peak loads. If write performance deteriorates, this would indicate the need to switch from the SQLite development configuration to the PostgreSQL production setup.

### **Reliability and Recoverability:**

The backend can be tested for correct state recovery by forcibly terminating the uvicorn process mid request during an order placement and then restarting it. Since SQLite only commits fully completed transactions, the database should reflect only the last committed state with no partial writes. For a production PostgreSQL deployment, this same principle would also apply. This would also involve verifying connection pool recovery on restart. To become reasonably confident in runtime reliability, we'd need to achieve continuous automated execution of the full test suite over several hours. This is with the assumption that no failures or state corruption is observed.

